

Setup

```
In [ ]: import numpy as np
import matplotlib.pyplot as plt

Z = (2*np.pi)**1.5 # this is also S_exact, funny coincidence
print(Z)
```

Metropolis sampler

```
In [ ]: def metropolis_chain(N, delta=1.7, burnin=2000, seed=0):
    rng = np.random.default_rng(seed)
    x = y = z = 0.0
    logw = -0.5*(x*x + y*y + z*z) # log-weight, avoids overflow
    total = burnin + N
    xs = np.empty(N); ys = np.empty(N); zs = np.empty(N)
    steps = rng.uniform(-delta, delta, size=(total, 3)) # all proposed
moves, pre-rolled
    u = rng.random(total) # coin flips: accept/reject
    n_accept = 0
    for i in range(total):
        xt, yt, zt = x+steps[i,0], y+steps[i,1], z+steps[i,2]
        logwt = -0.5*(xt*xt + yt*yt + zt*zt)
        if logwt ≥ logw or u[i] < np.exp(logwt-logw): # better point, or
got lucky
            x, y, z, logw = xt, yt, zt, logwt
            n_accept += 1
        if i ≥ burnin: # only keep stuff after warmup
            j = i - burnin
            xs[j], ys[j], zs[j] = x, y, z
    return xs, ys, zs, n_accept/total
```

Quick check

```
In [ ]: xs, ys, zs, acc = metropolis_chain(20000, seed=1)
f = (xs*ys*zs)**2
print(acc, Z*f.mean()) # acceptance rate, then our S guess
```

error vs N (10.3)

```
In [ ]: Ns = [1000, 2000, 4000, 8000, 16000, 32000, 64000, 128000]
        R = 60 # repeats per N, just to see the spread

        means, stds = [], []
        for N in Ns:
            ests = np.empty(R)
            for r in range(R):
                xs, ys, zs, _ = metropolis_chain(N, seed=1000*N + r)
                f = (xs*ys*zs)**2
                ests[r] = Z * f.mean()
            means.append(ests.mean())
            stds.append(ests.std(ddof=1))

        means = np.array(means); stds = np.array(stds)
        print(stds)
```

```
In [ ]: slope, _ = np.polyfit(np.log10(Ns), np.log10(stds), 1)
        print(slope) # should land near -0.5

        fig, ax = plt.subplots()
        ax.loglog(Ns, stds, 'o-')
        ax.loglog(Ns, stds[0]*(Ns[0]/np.array(Ns))**0.5, 'k--')
        fig.savefig("error_vs_N.png")
```

long chain + autocorrelation (10.4)

```
In [ ]: xs, ys, zs, acc = metropolis_chain(100000, delta=1.7, burnin=5000, seed=777)
        f = (xs*ys*zs)**2
        print(Z*f.mean(), Z*f.std(ddof=1)/np.sqrt(len(f))) # S estimate, naive
        error bar

        f_ = f.mean(); fc = f - f_; var = fc.var()
        kmax = 60
        acf = np.array([np.mean(fc[:len(fc)-k]*fc[k:]) for k in range(kmax)]) / var
        # correlation per lag
```

```
In [ ]: fig, ax = plt.subplots()
        ax.plot(range(kmax), acf, 'o-')
        ax.axhline(0)
        fig.savefig("acf.png")

        tau_int = 0.5 + acf[1:np.argmax(acf < 0)].sum() # rough correlation time
        skip = np.argmax(acf < 0.05) # where it's basically flat
        print(tau_int, skip)
```

effective N + thinning check

```
In [ ]: N_eff = len(f) / (2*tau_int)
        print(Z*f.std(ddof=1)/np.sqrt(N_eff)) # should line up better with the
        repeats

        thinned = f[::10] # keep every 10th
        print(Z*thinned.mean(), np.corrcoef(thinned[:-1], thinned[1:])[0,1])
```